Welcome to BLA 04, Part 2 of learning series for the AWS Machine Learning Engineer Associate exam. In Part 1, we chose where and how models should run. In this video we focus on provisioning and configuring the resources behind those decisions. We will cover on-demand versus provisioned capacity, infrastructure as code, containers, private networking for endpoints, auto scaling metrics, Amazon Bedrock Knowledge Bases, and the infrastructure that agentic workloads need. This maps to Task 3.2 of the exam guide. Let's jump in.

Every resource decision balances performance against cost. On-demand capacity has no commitment. You pay for what you use, which suits new or unpredictable workloads. In Amazon Bedrock, on-demand means paying per input and output token. Provisioned capacity reserves resources in advance. Bedrock Provisioned Throughput guarantees model capacity for steady traffic, and SageMaker AI Savings Plans lower the price when you commit to consistent usage. Choose this when traffic is predictable and performance must be consistent. The third option is Spot capacity for training. Managed Spot Training uses spare EC2 capacity at a big discount, but jobs can be interrupted. That is why checkpointing matters: the job saves progress so it can resume instead of starting over. Exam tip: steady and predictable usually points to provisioned, spiky or new points to on-demand, and cost-sensitive training that tolerates interruption points to Spot.

Clicking through the console does not scale and cannot be repeated reliably. So we provision with code. On the left is infrastructure as code. AWS CloudFormation describes resources in templates, and each deployment becomes a stack. The AWS Cloud Development Kit, or CDK, lets you write those stacks in Python or TypeScript. Real systems use several stacks, for example networking, storage, and the endpoint. Stacks share values through outputs and exports, or through AWS Systems Manager Parameter Store. On the right is programmatic deployment. The SageMaker Python SDK can deploy a model in one call to model dot deploy. With Boto3 or the AWS CLI, deployment is three calls: create the model, create the endpoint configuration, then create the endpoint. Remember that order for the exam. Finally, AWS Step Functions and Amazon EventBridge can orchestrate these steps so they run automatically.

SageMaker AI runs training and inference inside containers, and there are three levels of customization. Prebuilt containers, including the AWS Deep Learning Containers, already support frameworks like PyTorch, TensorFlow, XGBoost, and large model inference. AWS maintains them, and you usually just supply your own training or inference script. If you need one extra library, extend a prebuilt container. You write a Dockerfile that starts from the AWS image, add your dependencies, and push it to Amazon Elastic Container Registry. You keep AWS's base image and its updates. When nothing standard fits, bring your own container. For inference, the container must answer HTTP requests on port eight thousand eighty, with an invocations path for predictions and a ping path for health checks. Whichever option you pick, scan images for vulnerabilities and tag versions clearly. The exam usually rewards the least custom option that still works.

Many organizations require models to run inside a private network. With VPC configuration, you give the SageMaker AI model a list of subnets and security groups. Use private subnets in more

than one Availability Zone for resilience, and let security groups allow only the ports you need. This also lets the model reach private resources, like a database in the same VPC. Next, interface VPC endpoints powered by AWS PrivateLink let applications call the SageMaker Runtime API without traffic going over the public internet. A gateway endpoint for Amazon S3 lets the endpoint read model artifacts privately. Endpoint policies can restrict who uses them. Finally, network isolation blocks all outbound network calls from the container. That protects sensitive data, but the model must not depend on downloading anything at runtime. On the exam, words like no internet access or private connectivity point to these three settings.

Auto scaling only works well when it follows the right metric. For most real-time endpoints, the common choice is target tracking on invocations per instance, which is the SageMakerVariantInvocationsPerInstance metric. Run a load test to find how many requests one instance can handle, set the target below that, and always set minimum and maximum instance counts. Resource metrics are better when the bottleneck is hardware. For GPU workloads, like foundation models, GPU utilization and GPU memory utilization often show pressure before invocation counts do. With inference components, you can also scale on concurrent requests per model copy. For asynchronous endpoints, scale on the queue: the backlog size per instance. Async endpoints can scale to zero when the queue is empty, and a step scaling policy brings capacity back when new requests arrive. Exam tip: pick the metric that reflects the real limit of the workload.

Amazon Bedrock Knowledge Bases provide a managed RAG pipeline, and each step is a configuration choice. First, ingest documents from a data source, most often Amazon S3. Second, chunk them. Fixed-size chunking is simple. Semantic chunking splits on meaning. Hierarchical chunking keeps small chunks for precise matching but returns larger parent chunks for context. Third, embed each chunk into a vector with an embeddings model, such as Amazon Titan Text Embeddings. Fourth, store the vectors in a vector store, such as Amazon OpenSearch Serverless, Amazon Aurora with pgvector, or Amazon S3 Vectors. Fifth, retrieve. You set how many results come back, choose semantic or hybrid search, and optionally add reranking and metadata filters. The pipeline should match the business need. Precise policy questions favor smaller chunks and reranking, while broad summary questions need more context per result.

Agents add new infrastructure needs. The first is state. An agent must remember what happened earlier in a conversation or task. Amazon Bedrock Agents support session attributes that carry context between turns. Amazon Bedrock AgentCore Memory offers short-term memory for the current session and long-term memory across sessions. For custom designs, Amazon DynamoDB is a common choice to store agent state. The second need is GPU scaling. Foundation models behind agents need the right GPU instance size, and inference components can scale down, even to zero, when traffic drops. Watch GPU memory because large models run out of memory before they run out of compute. The third need is the agent infrastructure itself. AgentCore Runtime hosts agent code, the AgentCore Gateway can expose APIs and Lambda functions as MCP tools, and each tool should use a least-privilege IAM role.

Here is the checklist for Part 2. One: match on-demand, provisioned, or Spot capacity to how predictable the workload is. Two: define every stack and deployment in code, using CloudFormation, CDK, the SageMaker Python SDK, or Boto3. Three: use the least custom container that works, starting with prebuilt images. Four: place endpoints in a VPC with private access, and use network isolation when data is sensitive. Five: scale on the metric that reflects the real bottleneck, whether that is invocations, GPU utilization, or queue backlog. Six: configure knowledge bases carefully and manage agent state on purpose, because these choices directly affect answer quality. In Part 3, we will automate all of this with CI/CD pipelines and MLOps practices. Thanks for watching, and see you in the next one.